

SENA

TECNÓLOGO ANÁLISIS Y DESARROLLO DE SOFTWARE

MATERIA:

Fase 2. Planeación

Actividad de proyecto 4

Evidencia:

**GA4-220501095-AA2-EV01. Taller de conceptos y principios de programación
orientada a objetos**

Gerson Jonnathan López Oviedo – 3186647

DOCENTE:

Arnaldo Alfonso Montiel Brun

BOGOTÁ D.C.

6 de octubre del 2025

Introducción

En este taller abordo los conceptos fundamentales de la programación orientada a objetos (POO), una metodología que permite construir sistemas más organizados, reutilizables y escalables. A través de un glosario y una explicación personal de los principios clave, busco comprender y aplicar esta forma de programar que se basa en clases, objetos y relaciones entre ellos.

CONCEPTO PERSONAL SOBRE POO

Para mí, la programación orientada a objetos es una forma de organizar el código como si estuviéramos armando un mundo digital con personajes, acciones y reglas. Cada clase es como una especie, cada objeto es un individuo, y los métodos son lo que pueden hacer.

Lo que más valoro de POO es que:

- Me permite **reutilizar código** sin repetirlo.
- Me ayuda a **proteger los datos** y evitar errores.
- Me da **flexibilidad** para que el sistema crezca sin romperse.
- Me permite **pensar en bloques**, como si estuviera armando piezas de LEGO.

Aplicar estos principios me hace sentir que el código tiene lógica, estructura y propósito. No es solo escribir líneas, es construir soluciones que funcionan como un ecosistema bien diseñado.

GLOSARIO DE TÉRMINOS EN POO

- **Clase**

Una clase es como el plano de una casa. No es la casa en sí, pero define cómo será: cuántas habitaciones, qué color, qué materiales. En programación, la clase define qué atributos y comportamientos tendrá un objeto.

- **Objeto**

Un objeto es como una casa construida a partir de ese plano. Es una instancia real de la clase, con sus propios valores. Si la clase es “Perro”, el objeto puede ser “Firulais”, con nombre, edad y ladrido propio.

- **Método**

Un método es como una acción que el objeto puede realizar. Si el objeto es un televisor, sus métodos serían “encender”, “cambiar canal” o “subir volumen”. Son funciones que viven dentro de la clase.

- **Atributo**

Los atributos son las características del objeto. Si el objeto es un carro, sus atributos pueden ser “color”, “marca”, “velocidad”. Son los datos que describen al objeto.

- **Encapsulamiento**

Es como tener una caja fuerte dentro del objeto. Solo se puede acceder a los datos si tienes la llave correcta (un método autorizado). Esto protege la información y evita que se modifique sin control.

- **Abstracción**

La abstracción es como usar un control remoto sin saber cómo funciona por dentro. Solo ves los botones que necesitas. En programación, mostramos solo lo esencial del objeto y ocultamos lo complejo.

- **Herencia**

La herencia es como cuando un hijo recibe rasgos de sus padres. En POO, una clase puede heredar atributos y métodos de otra. Por ejemplo, si “Animal” tiene “respirar”, “Perro” lo hereda sin tener que volver a escribirlo.

- **Polimorfismo**

El polimorfismo es como tener un botón “encender” que funciona diferente en una licuadora, un televisor o una lámpara. En programación, un mismo método puede comportarse distinto según el objeto que lo use.

CONCLUSIONES

Después de realizar este taller sobre programación orientada a objetos, entendí que POO no es solo una técnica de codificación, sino una forma de pensar y organizar soluciones. Cada concepto como clases, objetos, herencia o polimorfismo tiene su lógica, y cuando los conectamos, el sistema se vuelve más claro, más seguro y más fácil de mantener.

Las analogías me ayudaron mucho a comprender lo que antes parecía complicado. Por ejemplo, ver una clase como el plano de una casa, o el encapsulamiento como una caja fuerte, me permitió visualizar mejor cómo funciona el código por dentro.

Además, aprendí que aplicar POO me da ventajas como:

- Reutilizar código sin repetirlo.
- Proteger los datos importantes.
- Adaptar comportamientos sin romper el sistema.
- Pensar en bloques que se conectan como piezas de LEGO.

Este conocimiento me motiva a seguir profundizando en el desarrollo de software, porque sé que dominar estos principios me permitirá construir aplicaciones más profesionales, limpias y escalables.